

Graphics - Visuals

Authors

Matt ter Steege
Matttersteege@gmail.com

Contents

1	voorkennis	2
1.1	Projecties	2
1.2	Kleuren	2
2	Ray Tracing	2
2.1	Schaduw	3
3	Materialen	3
3.1	diffuse materialen	3
3.2	glossy materialen	4
3.3	Ambient licht & Meerdere lichtbronnen	4
3.4	Reflectie	4
3.5	Refractie	4
4	Textures	5
4.1	Texture mapping	5
4.2	Image texture	5
4.3	Procedural Textures	5
5	Scene Graph	6
5.1	Object Space en World Space	6
5.2	Scene Graph	6
6	Optimalisaties	6
6.1	Acceleratiestructuur	6
6.2	Performance	7
7	Rasterisatie	7
7.1	De Rasterisatie Pipeline	7
7.2	De Rasterisatie Transformaties	8
7.3	OpenGL & GPU	9
7.4	Scene Graph (rasterisatie)	9
7.5	Vertex Shader	9
7.6	Fragment Shader	9
7.7	Zichtbaarheid	10

1 | voorkennis

Projecties

Er zijn 2 soorten projecties, Perspectief en Orthografisch. De eerste betekent dat er een camera (een punt) en een raster. Er wordt dan een ray “geschoten” vanuit het punt van de camera en dan door een pixel uit het raster en dan wordt er berekend of er iets geraakt wordt.

Orthografisch betekent dat de camera eigenlijk niet bestaat en dat de rays loodrecht staan op het raster. Er is dan geen diepte meer.

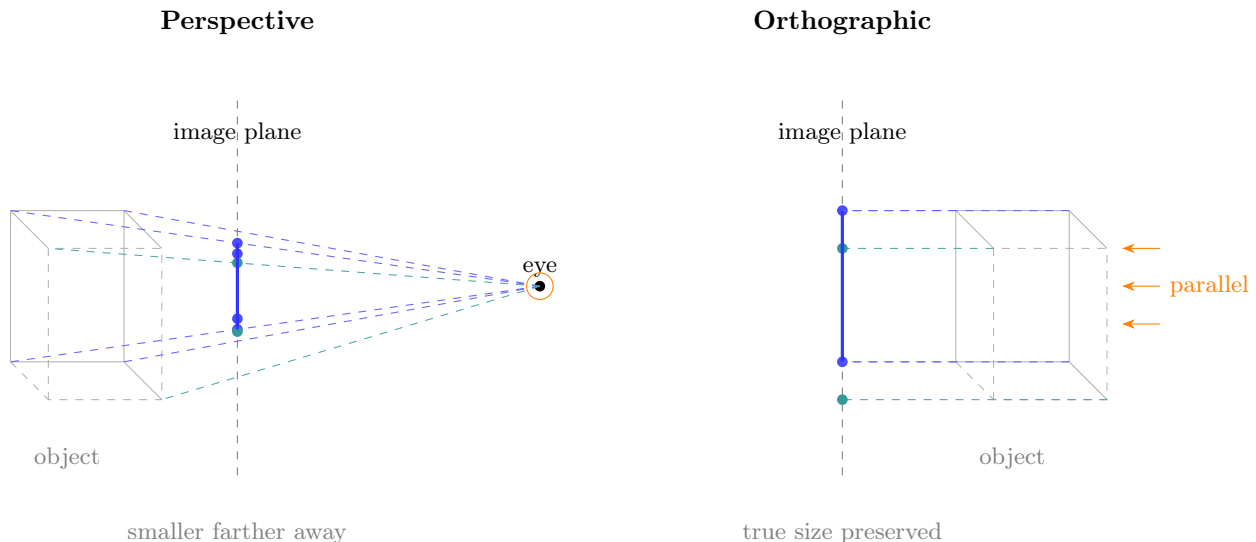


Figure 1: Perspective projection (left) uses a finite eye point so rays converge and distant objects appear smaller. Orthographic projection (right) uses parallel rays so the projected size is independent of depth.

Kleuren

Een computer heeft pixels, dat is niet nieuw en elke pixel bestaat uit een Rode, Groene en Blauwe subpixel. Deze kunnen individueel aan of uitgezet worden (of een waarde tussen aan en uit) om alle zichtbare kleuren te geven. RGB is het meest gebruikte format. Dit format heeft drie waarden, `int R`, `int G`, `int B`. Deze waarden lopen van 0 tot en met 255 (incl.) 255 is de maximale waarde die in 1 byte (8 bits) kan worden opgeslagen. Voorbeelden zijn:

- Zwart (0, 0, 0)		- Rood (255, 0, 0)	
- wit (255, 255, 255)		- Groen (0, 255, 0)	
		- Blauw (0, 0, 255)	

Intern worden RGB waarden ook vaak als een float waarde tussen 0...1 geschreven, waar 1 eigenlijk 255 betekent.

2 | Ray Tracing

Ray tracing is een renderingstechniek waarbij voor elke pixel een straal (*ray*) vanuit de camera door het beeldvlak de scène in wordt gestuurd. Voor elke straal wordt bepaald welk object als eerste wordt geraakt. Het snijpunt wordt vervolgens gebruikt om de kleur van de pixel te berekenen op basis van lichtbronnen, schaduwen en materiaaleigenschappen.

Het basialgoritme bestaat uit vier stappen:

1. Genereer een kijkstraal voor elke pixel.
2. Bepaal het dichtstbijzijnde snijpunt tussen de straal en de objecten in de scène.
3. Bereken de belichting op dat snijpunt, eventueel met behulp van schaduwstralen naar lichtbronnen.
4. Ken de berekende kleur toe aan de pixel.

Ray tracing gebruikt dus een “achterwaartse” benadering: in plaats van lichtstralen vanaf de lichtbron te volgen, worden stralen vanuit de camera verstuurd. Dit is veel efficiënter omdat alleen de zichtbare delen van de scène worden berekend. De techniek levert realistische beelden op met correcte zichtbaarheid, schaduwen, reflecties en andere lichteffecten.

Schaduw

Als er een lichtstraal gestuurd is vanuit de camera naar het object, dan wordt er op het intersectiepunt een “schaduwstraal” verstuurd dat vanuit dat punt direct naar alle lichtpunten in de scene wordt gestuurd. Het enige wat deze straal checkt is of er vanuit dit punt een directe weg is naar een lichtbron. Als er iets tussen zit dan krijgt dat punt geen licht en betekent dat dus direct: zwarte pixel. Als er wel een direct pad is, dan krijgt het object de kleur die het object zou moeten hebben.

3 | Materialen

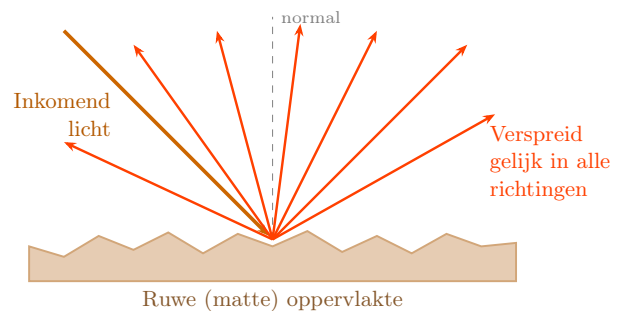
diffuse materialen

Bij diffuse materialen worden lichtstralen die op het materiaal vallen intern rond gehusseld en worden in een random richten weer weerkaats. Hierdoor is elke kant van het materiaal dezelfde kleur. De kleur hangt af van:

- De intensiteit (RGB) van het lichtpunt.
- De afstand van het lichtpunt
- De hoek waarin het licht aankomt
- De reflectie (RGB) op dat punt

De formule om dit uit te rekenen is als volgt:
 $C_{pixel_kleur} = L_{reflected} = I \frac{1}{r^2} \cdot \max(0, \hat{n} \cdot \hat{l}) \cdot k_d$. Dit ga ik even uitleggen (alles achter de tweede = dan).

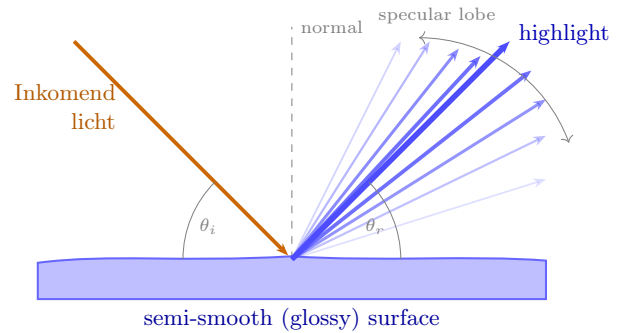
I is de uitgestuurde intensiteit van het lichtpunt, dit is een redelijk arbitraire waarde. $\frac{1}{r^2}$ komt van de “distance attenuation”. Dit betekent dat licht dat van een lichtpunt weggeschoten is uit elkaar spreidt, maar de totale energie blijft hetzelfde, dus dan moet de intensiteit afnemen naarmate het verder reist. Je kan het vergelijken met een glas water, als het water in het glas zit, dan is het water dichtbij elkaar en hoog in het glas. Neem je dezelfde hoeveelheid water en leg je het op een grote plaat, dan verspreid het en krijg je een dunne laag water. Dat is hetzelfde idee. r is de afstand tussen het lichtpunt en het raakpunt van het object. $(\hat{n} \cdot \hat{l})$ is het dotproduct van de **genormaliseerde** normaalvector op dat punt en de **genormaliseerde** richting van de lichtstraal. $\Rightarrow$



De laatste waarde is de k_d is de kleur die je aan het object gegeven hebt in RGB (0-1) dus bijvoorbeeld $[0, 0.8, 1]$ voor een turquoise kleur. Houdt er dus rekening mee dat C_{pixel_color} , $L_{reflected}$ en k_d RGB matrixen zijn (1×3).

glossy materialen

Ook voor glossy materialen is er een hele mooie formule: $C_{pixel_kleur} = L_{reflected} = I_{\frac{1}{r^2}} \cdot \max(0, \hat{v} \cdot \hat{r})^n \cdot k_s$. Ook deze wordt natuurlijk uitgelegd, de eerste termen zijn nog hetzelfde als bij diffuse materialen. $(\hat{v} \cdot \hat{r})^n$ $\hat{r}$ is de “hoofdluchtstraal” dit is in het figuur hiernaast de dikke blauwe pijl. Hierlangs wordt het meeste licht gereflecteerd. $\hat{v}$ gelijk aan de grootste uitschieter van de specular lobe, de lichtstraal die zo ver mogelijk weg is van de “hoofdluchtstraal”. Hoe dichter deze bij elkaar liggen hoe meer een object perfect reflecterend in plaats van glossy is. We kunnen deze glossyness ook nog aanpassen door de arbitraire n ; $n > 1$ aan te passen. Let wel op, glossy objecten reflecteren alleen licht en **geen** andere objecten in de scene.



Ambient licht & Meerdere lichtbronnen

Als je deze kennis samen gaat voegen, dan krijg je de volgende formule:

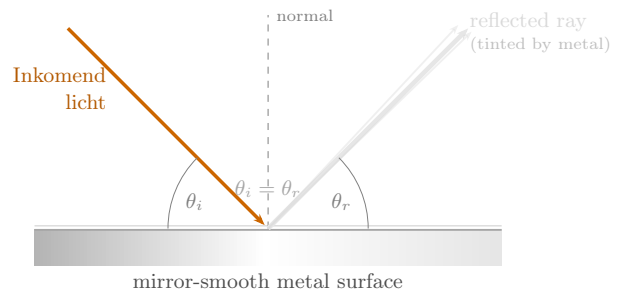
$C_{pixel_kleur} = L_{reflected} = I_{\frac{1}{r^2}} \cdot (\max(0, \hat{n} \cdot \hat{l}) \cdot k_d + \max(0, \hat{v} \cdot \hat{r})^n \cdot k_s) + k_a \cdot L_a$. Hier zijn nog twee extra termen bijgevoegd: k_a & L_a , de eerste geeft aan wat de ambiente waarde van het materiaal is (vaak gelijk aan k_d) en de tweede geeft aan welke kleur het licht is, dit is vaak een zachte grijze kleur.

Wil je nou verschillende lichtbronnen willen gebruiken, dan moet je de som nemen van alle termen behalve de $k_a \cdot L_a$, de uiteindelijke formule is dan:

$$C_{pixel_kleur} = L_{reflected} = [\sum_{i=1}^n I_i \frac{1}{r_i^2} \cdot (\max(0, \hat{n} \cdot \hat{l}_i) \cdot k_d + \max(0, \hat{v} \cdot \hat{r}_i)^n \cdot k_s)] + k_a \cdot L_a$$

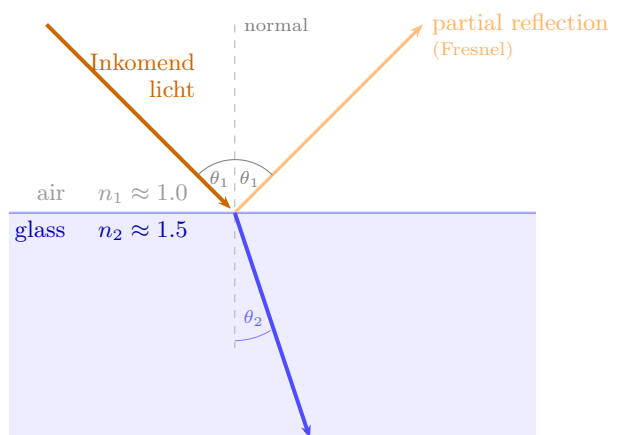
Reflectie

Een spiegel of reflecterend oppervlakte is eigenlijk niks meer dan een perfect glossy materiaal. Hierdoor kan je dus de term voor glossy herschrijven. $(\max(0, \hat{v} \cdot \hat{r}_i)^n \cdot k_s = k_m \cdot L_{mirror}) \Rightarrow (\hat{r} = \hat{v})$. Ofwel, wanneer $\hat{r} = \hat{v}$, dan kan je glossy herschrijven naar $k_m \cdot L_{mirror}$. Hier is k_m de kleur van de reflectie, dit kan dus bijvoorbeeld de kleur zijn van een muur als dat is wat een lichtstraal raakt nadat deze gereflecteerd is.



Refractie

Glas is nog eens heel speciaal, wanneer licht op een diëlektrisch materiaal valt (dus doorzichtig) dan gaat het licht door dat object heen, maar wel het verbuigt wel, als je een rietje in een glas water doet, dan gaat die ineens een andere kant op. Dat is refractie. Doorzichtige materialen hebben een “refractive index n ”. Deze index geeft aan hoe licht door dat materiaal voortbeweegt. Op zich betekend het niks, maar het is een waarde ten opzichte van een vacuüm (waar $n = 1$). $n_{lucht} \approx 1.000293$, $n_{water} \approx 1.333$ en $n_{glas} \approx 1.52$. Snell’s wet geeft aan wat de brekingshoeken zijn. $\frac{\sin \theta_2}{\sin \theta_1} = \frac{n_1}{n_2}$, hiermee kan je dus invoeren wat de invalshoek is, wat de indexen zijn van (1) het



huidige materiaal en (2) van het geraakte materiaal. Maar er *kan* ook een reflectie gebeuren, maar alleen als je van een hoge n materiaal naar een lage n gaat. Als hoek tussen de normaalvector van het glas groter wordt dan krijg je een totale interne reflectie en verlaat het licht het materiaal daar niet en wordt deze volledig gereflecteerd. De formule hiervoor gaat als volgt:

$$\hat{t} = \frac{n}{n_t}(\hat{d} - (\hat{d} \cdot \hat{n})\hat{n}) - \sqrt{1 - \frac{n^2}{n_t^2}(1 - (\hat{d} \cdot \hat{n})^2)}\hat{n}$$

Als de waarde onder de wortel < 0 is, dan heb je totale interne reflectie (dus al het licht reflecteert binnen het materiaal). n is de waarde van het huidige materiaal, n_t het geraakte materiaal. $\hat{d}$ is de inkomende lichtstraal. $\hat{n}$ is de normaalvector. $\hat{t}$ is de gereflecteerde lichtstraal. Ook is het goed om te weten dat je twee normaalvectoren gebruikt, die naar boven vanaf het raakpunt (voor de inkomende lichtstraal) en die naar benede voor de gereflecteerde lichtstraal.

4 | Textures

Texture mapping

Het idee van texture mapping is dat je een soort behang over een object heen legt. Dit “behang” kan een foto zijn of wiskunde worden berekend zoals een grid. Maar een texture is 2D, en moet op een 3D object worden gelegd. Dit is texture mapping en het werkt zo: stel dat je een vlak heb ($Ax + By + Cz + D = 0$, met de normaalvector $\vec{n} = (A, B, C)$). Er worden 2 vectoren $\vec{u}$ en $\vec{v}$ gebruiken om een punt op dat vlak aan te wijzen en dus een kleur terug te geven. Zo kan je dus uiteindelijk met de functie $F(u, v)$ een kleur aangeven. De driehoek hiernaast geeft aan hoe een driehoek mogelijk gemapt kan worden over de texture.

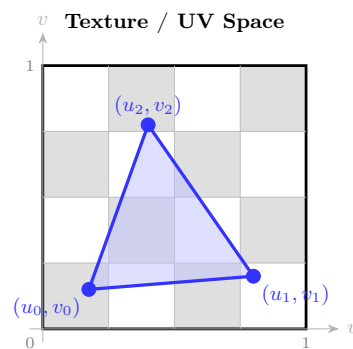
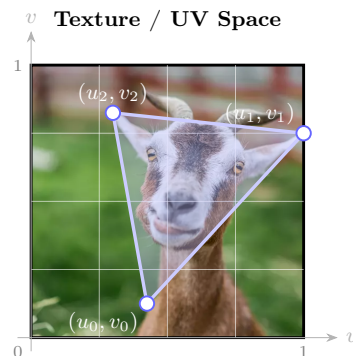


Image texture

Maar stel nou, je wilt een geweldige foto over die driehoek heen leggen. Dat kan ook, elke foto heeft een hoogte $h_{top} = v = 0$ en $h_{bottom} = v = 1$ en een breedte $b_{links} = u = 0$ en $b_{rechts} = u = 1$. Een mogelijke manier om die $F(u, v)$ functie te definiëren zou zijn: $F(u, v) = \text{image}[(\text{int})(u \times \text{breedte}), (\text{int})(v \times \text{hoogte})]$ om een pixel op een specifieke positie op de foto te krijgen.



Nu is het ook nog mogelijk dat een de u of v buiten de range $[0,1]$ ligt, dan kan je verschillende dingen doen, de waarde clampen (dus, $u' = \min(\max(u, 0), 1)$), de waarde is maximaal 1 en minimaal 0. Een andere optie is Repeat: $u' = u - \text{floor}(u)$, waar een waarde van $2.2 \rightarrow 0.2$ wordt. Of Mirror repeat, waar de foto logischerwijze steeds wordt gespiegeld. Een verdere toevoeging is Tiling en Offset, dat betekent dat je een foto misschien 3x op een foto wil hebben, dan kan je de $u' = n \times u$ doen, voor de hoeveelheid tiles in de u -kant op (dus links-rechts). Offset verplaatst de startpositie van de “nulwaarde”, deze is natuurlijk $u' = u + \text{offset}$.

Procedural Textures

Dit is super simpel, hier is de uv-map niet meer afhankelijk van een foto, maar van wiskunde. Zo kan je een formule voor een grid zo schrijven: $F(u, v) = (\text{floor}(u) + \text{floor}(v)) \bmod 2$. Andere opties zijn perlin noise of Voronoi noise (Google maar).

5 | Scene Graph

Wanneer je een scene opbouwt, wil je objecten op verschillende posities, rotaties en schalen plaatsen. Bij eenvoudige objecten zoals een bol, vlak of driehoek kan je de coördinaten rechtstreeks aanpassen. Bij een mesh met duizenden driehoeken zou je dan echter alle vertices moeten veranderen, wat veel rekenwerk kost.

Daarom wordt elk object gedefinieerd in zijn eigen *Object Space*. Vervolgens krijgt het object een *Object-to-World* transformatiematrix mee die bepaalt waar het object zich in de scene bevindt.

Een transformatie wordt opgeslagen in een 4x4 matrix:

$$\begin{bmatrix} \blacksquare & \blacksquare & \blacksquare & \blacksquare \\ \blacksquare & \blacksquare & \blacksquare & \blacksquare \\ \blacksquare & \blacksquare & \blacksquare & \blacksquare \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Deze matrix bevat de translatie (positie), rotatie en schaal van het object. In plaats van alle vertices van een mesh aan te passen, wordt de matrix gebruikt om het volledige object naar de juiste plaats in de wereld te transformeren.

Hierdoor kan hetzelfde object meerdere keren gebruikt worden met verschillende transformaties, zonder de geometrie te dupliceren.

Object Space en World Space

Alle objecten worden lokaal gemodelleerd in hun eigen Object Space, meestal rond de oorsprong (0,0,0). Tijdens het renderen moeten stralen (rays) worden getest tegen deze objecten. In plaats van het object naar World Space te transformeren, wordt de ray getransformeerd naar de Object Space van het object met behulp van de inverse matrix: M^{-1} . De intersectieberekening gebeurt vervolgens in Object Space. Het gevonden snijpunt en de normaalvector worden daarna terug naar World Space getransformeerd. Dit is efficiënter dan alle vertices van een mesh telkens naar World Space om te rekenen.

Scene Graph

Een scene graph is een boomstructuur waarin objecten en hun transformaties worden opgeslagen.

Elke node bevat:

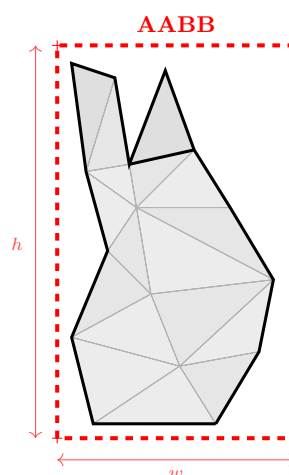
- een object (mesh, bol, vlak, ...)
- een transformatiematrix
- eventuele child-nodes

De transformatie van een child-node is relatief ten opzichte van zijn parent. De uiteindelijke positie van een object wordt bepaald door alle transformaties langs het pad van de root naar dat object te combineren. Hierdoor kunnen complexe objecten eenvoudig worden opgebouwd. Bijvoorbeeld een auto die bestaat uit een carrosserie, vier wielen en een geschutskoepel (kijk maar naar de slides als dit niet bekend is). Wanneer de auto verplaatst wordt, bewegen alle onderdelen automatisch mee.

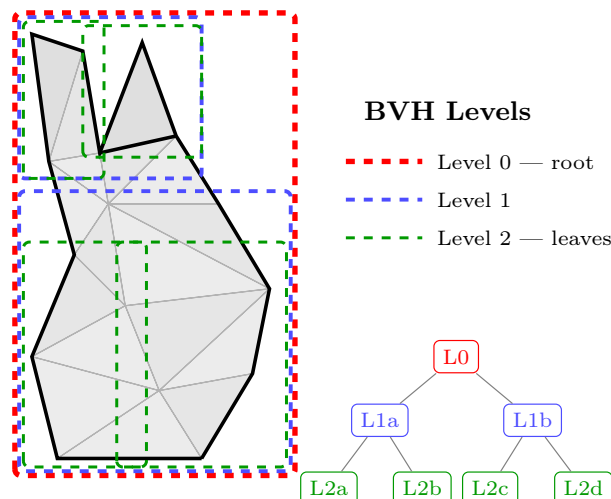
6 | Optimalisaties

Acceleratiestructuur

Het checken van intersecties met meshes (dus een heleboel driehoeken), duurt enorm lang. Een optie is een Axis-Aligned Bounding Box (AABB) dit is een kubus die volledig om een mesh heen gaat. Dan kan je eerst checken of er überhaupt een interactie is met de bounding box, gebeurd dat niet dan kan je alle driehoeken daarvan gelijk overslaan. Het kost even wat tijd om een AABB op te zetten, maar als het al één keer gebeurd dat je die hele AABB niet raakt, dan scheelt dat al meer tijd dan het in eerste instantie kostte.



Een andere optie is Bounding Volume Hierarchy (BVH) deze techniek is eigenlijk een paar AABB's in elkaar. Je checkt of een ray eerst de root raakt. Ja? dan check je alle branches, en daarna alles leaves. Dan heb je in (voor figuur 3) 3 checks al de meeste driehoeken uitgesloten, dan hoef je nog maar een stuk of 5 driehoeken daadwerkelijk te checken om te weten waar hij dan echt raakt. Deze 8 checks zijn veel sneller dan elke driehoek apart checken. Ook is een vierkant checken sneller dan een driehoek, dus dat levert ook tijd op.



Performance

Ik raad aan even Lecture 11 Dia 12-19 óók door te lezen

Om zogeheten “performance” te meten kan je de volgende C# code gebruiken:

```
using System.Diagnostics.Stopwatch;
Stopwatch timer = new Stopwatch();
timer.Start();
... do some work here ...
timer.Stop();
Console.WriteLine(timer.ElapsedMilliseconds);
timer.Reset();
```

Dit kan je gebruiken om stukken code in orde van milliseconden en groter te meten, om zekerder te zijn moet je dit meerdere keren meten, want achtergrondprocessen op je computer kunnen dit beïnvloeden.

Je stappenplan hiervoor is

1. Maak de niet-geoptimaliseerde applicatie.
2. Zoek bottlenecks, dus kijk op welke plekken je applicatie het langzaamst is en ga deze sneller proberen te maken.
3. Implementeer een mogelijke oplossing.
4. Meten is weten, welke is beter.
5. Houd de beste implementatie.

Voor de rest kan ik niks nuttigs vertellen, lees de slides maar!

7 | Rasterisatie

Rasterisatie is een alternatieve renderingstechniek ten opzichte van ray tracing. In plaats van stralen vanuit de camera te sturen, worden 3D-objecten via een reeks transformaties op het 2D-scherm geprojecteerd. Dit proces is zeer geschikt voor parallelisatie op een GPU.

De Rasterisatie Pipeline

De pipeline bestaat uit vier stappen:

1. **Transform Vertices:** Zet de 3D-vertices om naar 2D-schermcoördinaten + diepte via transformatiematrices.
2. **Discard Triangles:** Verbind vertices weer tot driehoeken en gooi driehoeken weg die buiten beeld vallen (clipping, backface culling).
3. **Rasterize:** Bepaal welke pixels door elke driehoek worden geraakt.
4. **Shade Fragments:** Bereken de kleur van elke geraakte pixel.

De Rasterisatie Transformaties

Het doel is simpel: je hebt een 3D-object en je wil dat op een 2D-scherm krijgen. Om dat te doen ga je het object door een reeks van stappen halen, elke stap is een transformatie (dus een matrixvermenigvuldiging).

Stap 1 — Modeling Transformation: het object in de wereld zetten

Je object is gemodelleerd rond zijn eigen oorsprong, dat heet *Object Space*. Een stoel staat bijvoorbeeld gewoon op $(0, 0, 0)$ in zijn eigen ruimte. Met de modeling transformation zeg je: “zet die stoel op positie $(3, 0, 2)$ in de wereld, roteer hem 45 graden en maak hem twee keer zo groot.” Dit is een gewone affine transformatie, dus gewoon rotatie + schaal + translatie gecombineerd in één matrix.

Stap 2 — Camera Transformation: de wereld vanuit de camera bekijken

Nu staat alles op de juiste plek in de wereld, maar je kijkt er nog niet naar. Je moet een camera definiëren. Die camera heeft drie dingen nodig:

- Waar staat hij? (positie $\vec{e}$)
- Waar kijkt hij naartoe? (de “look-at” vector $\vec{g}$)
- Welke kant is boven? (de “up” vector $\vec{t}$, meestal gewoon omhoog)

Vanuit die drie dingen bouw je een nieuw coördinatenstelsel voor de camera. Je berekent drie loodrechte assen: één die uit het scherm wijst ($\hat{w}$), één naar rechts ($\hat{u}$) en één omhoog ($\hat{v}$). Die bereken je met kruisproducten:

$$\hat{w} = -\frac{\vec{g}}{\|\vec{g}\|}, \quad \hat{u} = \frac{\vec{t} \times \hat{w}}{\|\vec{t} \times \hat{w}\|}, \quad \hat{v} = \hat{w} \times \hat{u}$$

Dan doe je twee dingen: je verschuift de hele wereld zodat de camera op de oorsprong staat, en je roteert alles zodat de camerarichtingen uitkomen met de gewone x -, y -, z -assen. Combineer die twee en je hebt $M_{\text{camera}} = RT$.

Het resultaat: alle objecten zitten nu in *Camera Space*, alsof de camera altijd op $(0, 0, 0)$ staat en recht vooruit kijkt.

Stap 3 — Projectie: van 3D naar 2D

Nu sta je in Camera Space, maar je hebt nog steeds 3D-coördinaten. Je moet die platdrukken op een 2D-vlak. Hier zijn twee opties:

Orthografisch is de simpele versie: je schaal alles netjes in een blok van -1 tot 1 in alle richtingen. Objecten ver weg zijn even groot als objecten dichtbij, want er is geen perspectief.

Perspectief is de realistische versie: dingen ver weg moeten kleiner lijken. Dit werkt via gelijkzijdige driehoeken. Als iets op afstand z staat en hoogte y heeft, dan zie je het op het scherm op hoogte:

$$y_s = \frac{n}{z} \cdot y$$

waarbij n de afstand tot je *near plane* is. Hoe verder weg, hoe groter de deler, hoe kleiner het ding op het scherm.

Dit kan je niet schrijven als een gewone affine transformatie, maar het kan wél als een 4×4 matrix:

$$P = \begin{bmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & n+f & -fn \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Na de vermenigvuldiging moet je nog delen door de vierde coördinaat (de w -waarde), dat is de zogenaamde *perspectieve deling*. Na de projectie zit alles in een blok van -1 tot 1 in x , y en z . Dat blok heet het *Canonical View Volume*.

Stap 4 — Viewport Transformation: van $[-1, 1]$ naar pixels

Het canonical view volume loopt van -1 tot 1 , maar je scherm heeft bijvoorbeeld 1920×1080 pixels. De viewport transformation schaal en verschuift de x - en y -coördinaten zodat ze kloppen met de pixelcoördinaten van je scherm. De z -waarde (diepte) blijft bewaard voor de z -buffer later.

Alles samen

Elke transformatie is een matrix. Je kan alle vier de matrices van tevoren met elkaar vermenigvuldigen tot één grote matrix M :

$$M = M_{\text{viewport}} \cdot M_{\text{projection}} \cdot M_{\text{camera}} \cdot M_{\text{model}}$$

Dan hoef je voor elke vertex maar één matrixvermenigvuldiging te doen om hem direct van Object Space naar schermcoördinaten te brengen. Dat is precies wat de **Vertex Shader** op de GPU doet, voor alle vertices tegelijk en parallel.

OpenGL & GPU

De GPU is een *streaming processor*: hij voert heel veel identieke taken tegelijk uit (één per vertex of fragment). In tegenstelling tot de CPU heeft hij veel meer kernen, maar ze moeten allemaal dezelfde code uitvoeren en hij gebruikt geen caches.

In moderne OpenGL wordt data maar één keer naar de GPU geüpload (retained mode) en worden de vertex- en fragmentshaders door de programmeur zelf geschreven in GLSL. De data wordt opgeslagen in een **Vertex Buffer Object (VBO)** en de opbouw ervan wordt vastgelegd in een **Vertex Array Object (VAO)**.

Scene Graph (rasterisatie)

Net als bij ray tracing gebruik je een scene graph als boomstructuur. Het verschil is dat je bij rasterisatie *geen* inverse matrix nodig hebt. In plaats daarvan geef je aan elk object twee matrices mee:

- **Object-to-Screen**: Om het object op het scherm te projecteren (inclusief perspectief).
- **Object-to-World**: Om shading-berekeningen in World Space te kunnen doen in de Fragment Shader.

Vertex Shader

De vertex shader draait op de GPU en verwerkt elke vertex afzonderlijk. De verplichte output is `gl_Position`, de positie in 2D Screen Space + diepte. Daarnaast worden andere waarden doorgegeven aan de fragment shader, zoals de positie en normaalvector in World Space.

Een voorbeeld in GLSL:

```
uniform mat4 objectToScreen;
uniform mat4 objectToWorld;
in vec3 vertexPositionObject;
in vec3 vertexNormalObject;
out vec4 positionWorld;
out vec4 normalWorld;

void main() {
    gl_Position = objectToScreen * vec4(vertexPositionObject, 1.0);
    positionWorld = objectToWorld * vec4(vertexPositionObject, 1.0);
    normalWorld = inverse(transpose(objectToWorld))
        * vec4(vertexNormalObject, 0.0);
}
```

Let op: normaalvectoren worden getransformeerd met de *inverse transpose* van de Object-to-World matrix, omdat gewone schaling de normalen scheef kan trekken (zie boek sectie 6.2.2).

Fragment Shader

De fragment shader berekent de kleur van elk fragment (pixel-kandidaat). De inputs zijn geïnterpoleerde waarden vanuit de vertex shader. De shading-berekeningen vinden plaats in World Space.

Diffuse shading

Identiek aan de formule uit ray tracing:

$$C = I \cdot \frac{1}{r^2} \cdot \max(0, \hat{n} \cdot \hat{l}) \cdot k_d$$

In GLSL:

```
vec3 l = lightPositionWorld - positionWorld.xyz;
float attenuation = 1.0 / dot(l, l);
float ndotl = max(0, dot(normalize(normalWorld.xyz), normalize(l)));
vec3 diffuseColor = texture(tex, uv).rgb;
color = vec4(lightColor * diffuseColor * attenuation * ndotl, 1.0);
```

Phong shading

De volledige Phong formule voegt speculaire en ambiente bijdragen toe:

$$C = I \cdot \frac{1}{r^2} \cdot \left(\max(0, \hat{n} \cdot \hat{l}) \cdot k_d + \max(0, \hat{v} \cdot \hat{r})^n \cdot k_s \right) + k_a \cdot L_a$$

Hierbij is $\hat{v}$ de richting naar de camera, $\hat{r}$ de gereflecteerde lichtrichting, en n de “shininess” waarde.

Reflecties & Environment Mapping

Spiegelreflecties van de omgeving zijn moeilijk in rasterisatie. Een oplossing is *environment mapping*: sla een foto van de omgeving op in een cube map of sphere map. In de fragment shader bereken je $\hat{r} = \vec{l} - 2(\vec{l} \cdot \hat{n})\hat{n}$ en gebruik je die als opzoekrichting in de texture. Dit is een benadering: je reflecteert alleen de achtergrond, niet andere objecten in de scene.

Normal Mapping

Een normal map is een texture waarin per texel een normaalvector is opgeslagen (als RGB-kleur, waarbij $xyz \in [-1, 1]$ wordt opgeslagen als $rgb \in [0, 255]$). De opgeslagen normalen zitten in *tangent space* (het lokale coördinatenstelsel op het oppervlak). Dit geeft de illusie van veel meer geometrisch detail dan er werkelijk is, zonder extra polygonen.

Zichtbaarheid

Painter's Algorithm

Sorteer driehoeken op diepte en render van achter naar voor. Nadelen: duur om te sorteren, werkt niet in alle gevallen en overdraw (pixels worden meerdere keren overschreven).

Z-buffer

De standaardoplossing in moderne rasterisatie. Per pixel wordt de diepte van het dichtste fragment bijgehouden. Een nieuw fragment wordt alleen getekend als het dichterbij de camera is dan de waarde in de z-buffer.

- Vereist een extra buffer naast de kleur-framebuffer
- Buffer wordt geïnitieerd op $z_{\max}$
- De diepte z wordt geïnterpoleerd over de driehoek
- Per fragment: lees z-buffer, vergelijk, schrijf eventueel terug

Clipping & Culling

Driehoeken die buiten het beeld of buiten het $[n, f]$ -bereik vallen worden weggegooid door *clipping*. Dit gebeurt in het Canonical View Volume (ook wel Clip Space). Op grovere schaal worden grote stukken van de scene al van tevoren verwijderd via frustum culling, occlusion culling, of portals. Per driehoek kan *backface culling* worden toegepast: driehoeken waarvan de achterkant naar de camera is gericht worden weggegooid.